Taller 3 – plan de muestreo estándar

Diego Fernando Muñoz Portela 2415620, Valery Rivera Lopez 2417038, Alejandro Barr

13 de marzo de 2026

Contents

1 Punto 1 **1**

1.1 Enunciado 1

1.2 Preguntas 2

1.2.1 a. ¿Cuál es el plan de muestreo simple adecuado bajo los niveles de inspección I, II, III, en los niveles normal, reducido y severo?. Evalúe la conveniencia de los planes resultantes. (OC, AOQ, ATI). 2

1.2.2 b. Ubique ahora los respectivos planes de muestreo doble. Evalúe la conveniencia de los planes resultantes. (OC, AOQ, ATI). 9

1 Punto 1

1.1 Enunciado

El jefe de compras de una empresa ha exigido un AQL de 2.5% para la característica de un cierto producto que se recibe en lotes de 3000 unidades y se desea obtener un plan de inspección por muestreo usando las normas Militar Standard.

Por otro lado el departamento de producción no está dispuesto a tolerar ningún lote con más del 10% de unidades defectuosas.

Como un lote se considera bueno si tiene menos del 5%, se tomará un tamaño de muestra n y se aceptará el lote si tiene menos del 5% de unidades defectuosas, por ejemplo:

- $(n = 100, c = 5)$
- $(n = 80, c = 4)$
- $(n = 40, c = 2)$

1.2 Preguntas

1.2.1 a. ¿Cuál es el plan de muestreo simple adecuado bajo los niveles de inspección I, II, III, en los niveles normal, reducido y severo?. Evalúe la conveniencia de los planes resultantes. (OC, AOQ, ATI).

```
#library(AQLSchemes)
#library(AcceptanceSampling)

#planS_1 <- AASingle('Normal')
#planS_1

#planS_2<-AASingle('Normal')
#planS_2


#planS_3<-AASingle('Normal')
#planS_3

#planE_1<-AASingle('Tightened')
#planE_1
#planE_2<-AASingle('Tightened')
#planE_2
#planE_3<-AASingle('Tightened')
#planE_3

#planr_1<-AASingle('Reduced')
#planr_1
#planr_2<-AASingle('Reduced')
```

```r
#planr_2
#planr_3<-AASingle('Reduced')
#planr_3

#planDN_1<-AADouble('Normal')
#planDN_1
#planDN_2<-AADouble('Normal')
#planDN_2
#planDN_3<-AADouble('Normal')
#planDN_3

# Crear el data frame sin columna Plan
tabla_simple <- data.frame(
  Nivel = c("I", "II", "III",
            "I", "II", "III",
            "I", "II", "III"),
  n = c(50, 125, 200, 50, 125, 200, 20, 50, 80),
  c = c(3, 7, 10, 2, 5, 8, 1, 3, 5),
  r = c(4, 8, 11, 3, 6, 9, 4, 6, 8)
)

library(knitr)
library(kableExtra)

tabla_simple %>%
  kbl(
    caption = "Planes de muestreo simple según nivel de inspección",
    col.names = c("Nivel", "n", "c", "r"),
    align = "c",
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover"),
    position = "center",
    font_size = 12  # Reducido para mejor ajuste
  ) %>%
```

Table 1: Planes de muestreo simple según nivel de inspección

	Parámetros del plan		
Nivel	n	c	r
Normal			
I	50	3	4
II	125	7	8
III	200	10	11
Estricto			
I	50	2	3
II	125	5	6
III	200	8	9
Reducido			
I	20	1	4
II	50	3	6
III	80	5	8

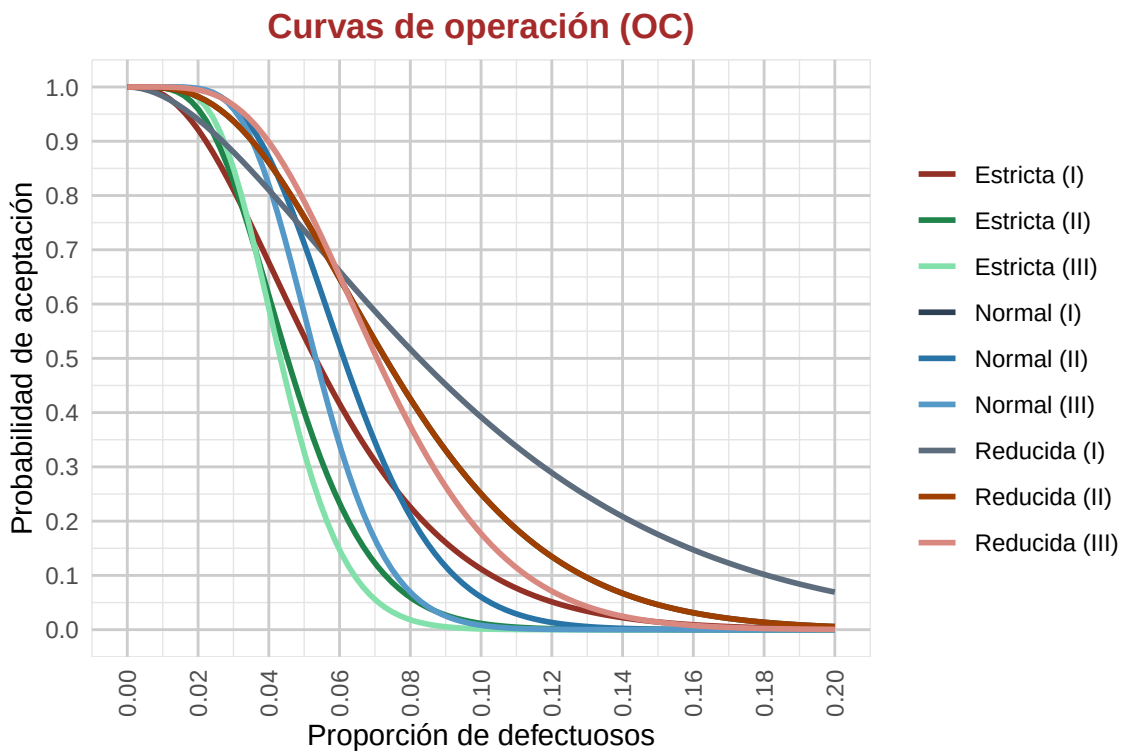
Note:
n: tamaño de muestra; c: número de aceptación; r: número de rechazo

```
  group_rows("Normal", 1, 3, bold = TRUE, background = "#f0f0f0") %>%
  group_rows("Estricto", 4, 6, bold = TRUE, background = "#f0f0f0") %>%
  group_rows("Reducido", 7, 9, bold = TRUE, background = "#f0f0f0") %>%
  column_spec(1:4, width = "2cm") %>%  # Anchos fijos iguales
  add_header_above(c(" " = 1, "Parámetros del plan" = 3)) %>%
  footnote(general = "n: tamaño de muestra; c: número de aceptación; r: número de rechaz
```

```
library(AcceptanceSampling)
library(tidyverse)

# 1. Definir la tabla de planes (ajustada para que coincida con la lógica de la imagen
tabla_planes <- tribble(
  ~Tipo,      ~Plan, ~n,  ~c,
  "Normal",   "I",   50,  3,
  "Estricta", "I",   50,  2,
  "Reducida", "I",   20,  1,
  "Normal",   "II",  125, 7,
  "Estricta", "II",  125, 5,
```

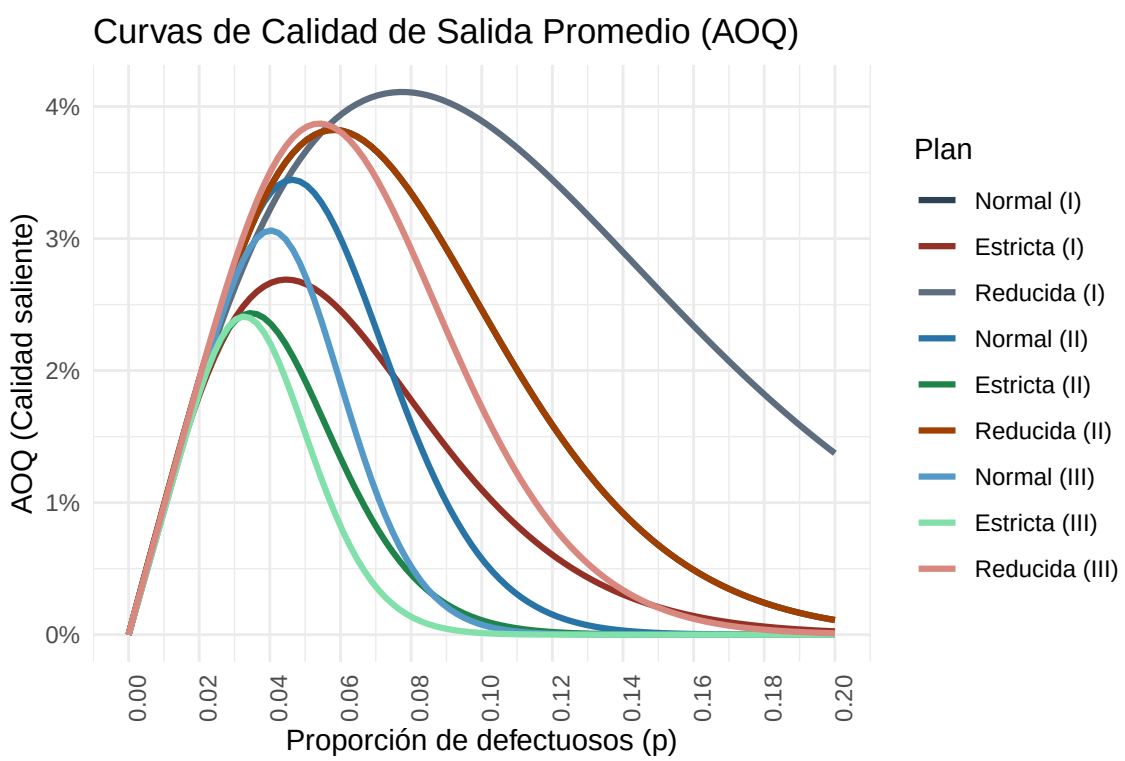
```r
  "Reducida", "II",  50,  3,
  "Normal",   "III", 200, 10,
  "Estricta", "III", 200, 8,
  "Reducida", "III", 80,  5
)

pd_vector <- seq(0, 0.20, length.out = 100)
datos_grafico <- tabla_planes %>%
  group_by(Tipo, Plan) %>%
  do({
    plan <- OC2c(n = .$n, c = .$c, type = "binomial", pd = pd_vector)
    tibble(pd = pd_vector, pa = plan@paccept)
  }) %>%
  ungroup() %>%
  mutate(Etiqueta = paste0(Tipo, " (", Plan, ")"))
colores_manuales <- c(
  "Normal (I)"   = "#2E4053", "Estricta (I)" = "#943126", "Reducida (I)" = "#5D6D7E",
  "Normal (II)"  = "#2874A6", "Estricta (II)"= "#1E8449", "Reducida (II)" = "#A04000",
  "Normal (III)" = "#5499C7", "Estricta (III)"= "#82E0AA", "Reducida (III)"= "#D98880"
)

ggplot(datos_grafico, aes(x = pd, y = pa, color = Etiqueta)) +
  geom_line(size = 1) +
  scale_x_continuous(breaks = seq(0, 0.2, 0.02), limits = c(0, 0.2)) +
  scale_y_continuous(breaks = seq(0, 1, 0.1), limits = c(0, 1)) +
  scale_color_manual(values = colores_manuales) +
  labs(
    title = "Curvas de operación (OC)",
    x = "Proporción de defectuosos",
    y = "Probabilidad de aceptación",
    color = NULL
  ) +
  theme_minimal() +
  theme(
    panel.grid.major = element_line(color = "grey80"),
    panel.grid.minor = element_line(color = "grey90"),
```

```
    axis.text.x = element_text(angle = 90, vjust = 0.5),
    legend.position = "right",
    plot.title = element_text(hjust = 0.5, face = "bold", color = "brown")
  )
```



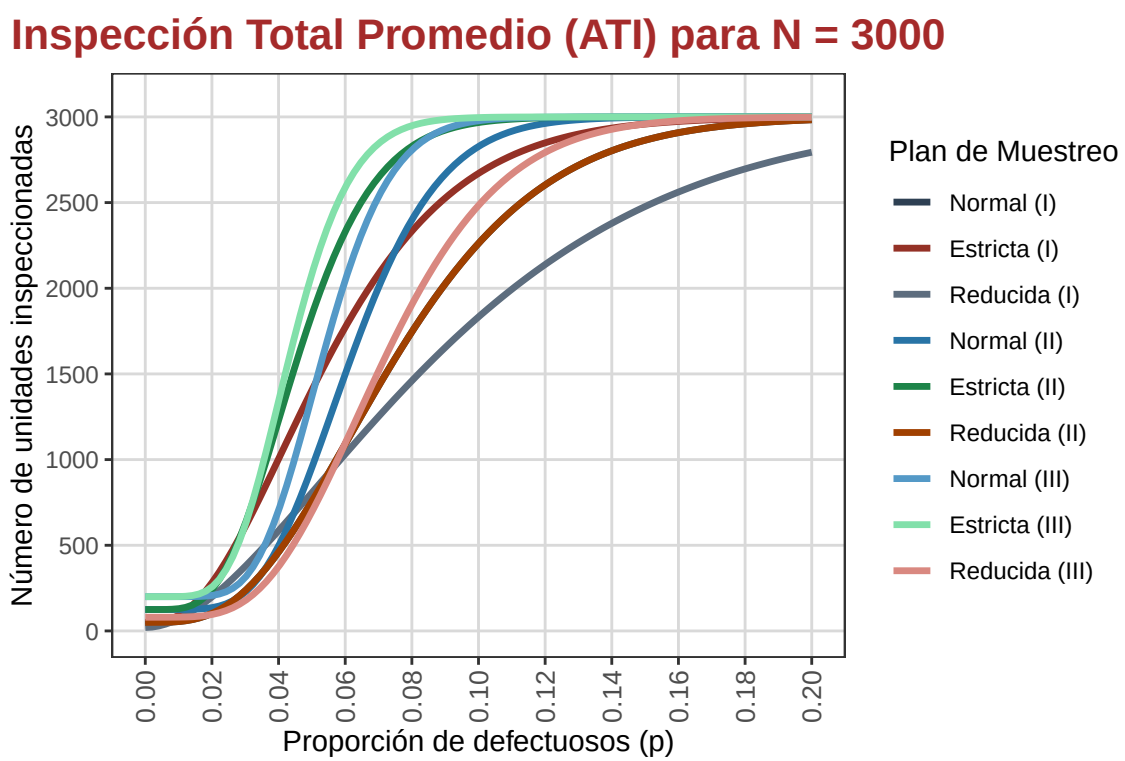

```
# Calcular AOQ
N_lote <- 3000
datos_aoq <- tabla_planes %>%
  group_by(Tipo, Plan) %>%
  do({
    plan <- OC2c(n = .$n, c = .$c, type = "binomial", pd = pd_vector)
    aoq_calculado <- plan@paccept * pd_vector * ((N_lote - .$n) / N_lote)
    tibble(pd = pd_vector, aoq = aoq_calculado)
  }) %>%
  ungroup() %>%
  mutate(Etiqueta = factor(paste0(Tipo, " (", Plan, ")"),
                           levels = names(colores_manuales)))
ggplot(datos_aoq, aes(x = pd, y = aoq, color = Etiqueta)) +
  geom_line(size = 1.1) +
```

```r
  scale_x_continuous(breaks = seq(0, 0.2, 0.02)) +
  scale_y_continuous(labels = scales::percent) +
  scale_color_manual(values = colores_manuales) +
  labs(title = "Curvas de Calidad de Salida Promedio (AOQ)",
       x = "Proporción de defectuosos (p)",
       y = "AOQ (Calidad saliente)",
       color = "Plan") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90))
```




```r
#ati
N_lote <- 3000

datos_ati <- tabla_planes %>%
  group_by(Tipo, Plan) %>%
  do({
    plan <- OC2c(n = .$n, c = .$c, type = "binomial", pd = pd_vector)
    ati_val <- .$n + (1 - plan@paccept) * (N_lote - .$n)
    tibble(pd = pd_vector, ati = ati_val)
```

```r
  }) %>%
  ungroup() %>%
  mutate(Etiqueta = factor(paste0(Tipo, " (", Plan, ")"),
                           levels = names(colores_manuales)))
ggplot(datos_ati, aes(x = pd, y = ati, color = Etiqueta)) +
  geom_line(size = 1.2) +
  scale_y_continuous(breaks = seq(0, 3000, 500), limits = c(0, 3100)) +
  scale_x_continuous(breaks = seq(0, 0.20, 0.02)) +
  scale_color_manual(values = colores_manuales) +
  labs(
    title = paste("Inspección Total Promedio (ATI) para N =", N_lote),
    x = "Proporción de defectuosos (p)",
    y = "Número de unidades inspeccionadas",
    color = "Plan de Muestreo"
  ) +
  theme_bw() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold", color = "brown"),
    axis.text.x = element_text(angle = 90, vjust = 0.5, size = 10),
    panel.grid.major = element_line(color = "grey85"),
    panel.grid.minor = element_blank(),
    legend.position = "right"
  )
```

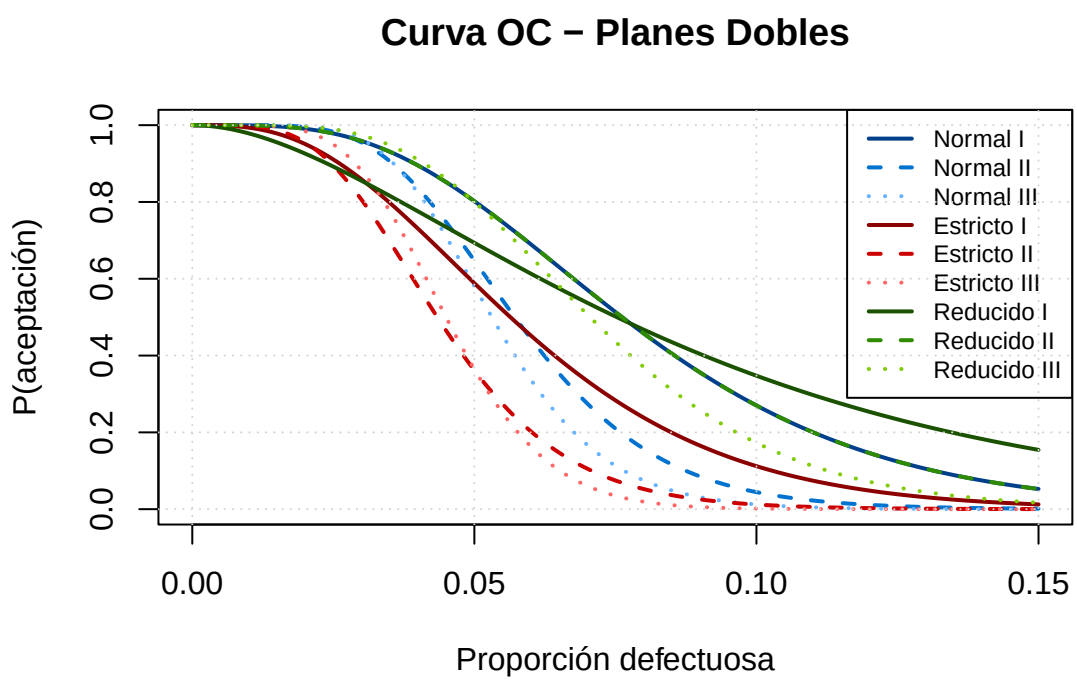


1.2.2 b. Ubique ahora los respectivos planes de muestreo doble. Evalúe la conveniencia de los planes resultantes. (OC, AOQ, ATI).

```
pd <- seq(0, 0.15, 0.001)

# Función para calcular Pa doble
Pa_doble <- function(n1, c1, r1, n2, c2, pd) {
  Pa1 <- pbinom(c1, n1, pd)
  Pa2 <- numeric(length(pd))
  for (i in seq_along(pd)) {
    pa2_i <- 0
    for (d1 in (c1 + 1):(r1 - 1)) {
      pa2_i <- pa2_i + dbinom(d1, n1, pd[i]) * pbinom(c2 - d1, n2, pd[i])
    }
    Pa2[i] <- pa2_i
  }
  return(Pa1 + Pa2)
}
```

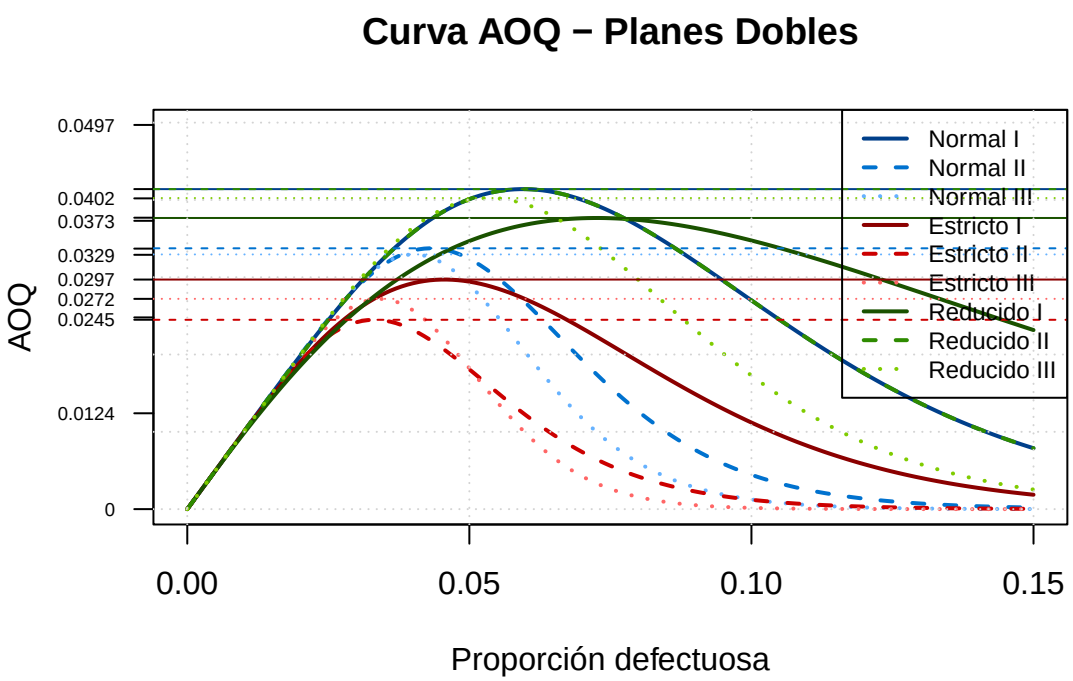
```r
#-------------------------------------#
# Normal
#-------------------------------------#
Pa_NI_Nor   <- Pa_doble(n1=32,  c1=1, r1=4, n2=32,  c2=4,  pd)
Pa_NII_Nor  <- Pa_doble(n1=80,  c1=3, r1=7, n2=80,  c2=8,  pd)
Pa_NIII_Nor <- Pa_doble(n1=125, c1=5, r1=9, n2=125, c2=12, pd)

#-------------------------------------#
# Estricto
#-------------------------------------#
Pa_NI_Est   <- Pa_doble(n1=32,  c1=0, r1=3, n2=32,  c2=3,  pd)
Pa_NII_Est  <- Pa_doble(n1=80,  c1=2, r1=5, n2=80,  c2=6,  pd)
Pa_NIII_Est <- Pa_doble(n1=125, c1=3, r1=7, n2=125, c2=11, pd)

#-------------------------------------#
# Reducido
#-------------------------------------#
Pa_NI_Red   <- Pa_doble(n1=13, c1=0, r1=4, n2=13, c2=1, pd)
Pa_NII_Red  <- Pa_doble(n1=32, c1=1, r1=4, n2=32, c2=4, pd)
Pa_NIII_Red <- Pa_doble(n1=50, c1=2, r1=6, n2=50, c2=6, pd)

azules <- c("#003f8a", "#0072CF", "#66B2FF")
rojos  <- c("#8B0000", "#CC0000", "#FF6666")
verdes <- c("#1a5200", "#2E8B00", "#7FCC00")

plot(pd, Pa_NI_Nor, type='l', lwd=2, lty=1, col=azules[1],
     main="Curva OC - Planes Dobles",
     xlab="Proporción defectuosa", ylab="P(aceptación)", ylim=c(0,1))

# Normal
lines(pd, Pa_NII_Nor,  lwd=2, lty=2, col=azules[2])
lines(pd, Pa_NIII_Nor, lwd=2, lty=3, col=azules[3])

# Estricto
lines(pd, Pa_NI_Est,   lwd=2, lty=1, col=rojos[1])
```

```r
lines(pd, Pa_NII_Est,  lwd=2, lty=2, col=rojos[2])
lines(pd, Pa_NIII_Est, lwd=2, lty=3, col=rojos[3])

# Reducido
lines(pd, Pa_NI_Red,   lwd=2, lty=1, col=verdes[1])
lines(pd, Pa_NII_Red,  lwd=2, lty=2, col=verdes[2])
lines(pd, Pa_NIII_Red, lwd=2, lty=3, col=verdes[3])

legend("topright",
       legend=c("Normal I","Normal II","Normal III",
                "Estricto I","Estricto II","Estricto III",
                "Reducido I","Reducido II","Reducido III"),
       col=c(azules, rojos, verdes),
       lwd=2, lty=c(1,2,3, 1,2,3, 1,2,3), cex=0.75)
grid()
```




```r
pd <- seq(0, 0.15, 0.001)

# Función para calcular Pa doble
```

```r
Pa_doble <- function(n1, c1, r1, n2, c2, pd) {
  Pa1 <- pbinom(c1, n1, pd)
  Pa2 <- numeric(length(pd))
  for (i in seq_along(pd)) {
    pa2_i <- 0
    for (d1 in (c1 + 1):(r1 - 1)) {
      pa2_i <- pa2_i + dbinom(d1, n1, pd[i]) * pbinom(c2 - d1, n2, pd[i])
    }
    Pa2[i] <- pa2_i
  }
  return(Pa1 + Pa2)
}

#-------------------------------------#
# Normal
#-------------------------------------#
Pa_NI_Nor   <- Pa_doble(n1=32,  c1=1, r1=4, n2=32,  c2=4,  pd)
Pa_NII_Nor  <- Pa_doble(n1=80,  c1=3, r1=7, n2=80,  c2=8,  pd)
Pa_NIII_Nor <- Pa_doble(n1=125, c1=5, r1=9, n2=125, c2=12, pd)

#-------------------------------------#
# Estricto
#-------------------------------------#
Pa_NI_Est   <- Pa_doble(n1=32,  c1=0, r1=3, n2=32,  c2=3,  pd)
Pa_NII_Est  <- Pa_doble(n1=80,  c1=2, r1=5, n2=80,  c2=6,  pd)
Pa_NIII_Est <- Pa_doble(n1=125, c1=3, r1=7, n2=125, c2=11, pd)

#-------------------------------------#
# Reducido
#-------------------------------------#
Pa_NI_Red   <- Pa_doble(n1=13, c1=0, r1=4, n2=13, c2=1, pd)
Pa_NII_Red  <- Pa_doble(n1=32, c1=1, r1=4, n2=32, c2=4, pd)
Pa_NIII_Red <- Pa_doble(n1=50, c1=2, r1=6, n2=50, c2=6, pd)

# Calcular los AOQL
AOQL_NI_Nor   <- max(Pa_NI_Nor*pd)
```

```r
AOQL_NII_Nor  <- max(Pa_NII_Nor*pd)
AOQL_NIII_Nor <- max(Pa_NIII_Nor*pd)
AOQL_NI_Est   <- max(Pa_NI_Est*pd)
AOQL_NII_Est  <- max(Pa_NII_Est*pd)
AOQL_NIII_Est <- max(Pa_NIII_Est*pd)
AOQL_NI_Red   <- max(Pa_NI_Red*pd)
AOQL_NII_Red  <- max(Pa_NII_Red*pd)
AOQL_NIII_Red <- max(Pa_NIII_Red*pd)

aoql_vals <- c(AOQL_NI_Nor, AOQL_NII_Nor, AOQL_NIII_Nor,
               AOQL_NI_Est, AOQL_NII_Est, AOQL_NIII_Est,
               AOQL_NI_Red, AOQL_NII_Red, AOQL_NIII_Red)

plot(pd, Pa_NI_Nor*pd, type='l', lwd=2, lty=1, col=azules[1],
     main="Curva AOQ - Planes Dobles",
     xlab="Proporción defectuosa", ylab="AOQ",
     ylim=c(0, max(aoql_vals)*1.2),
     yaxt="n")  # Desactiva el eje Y automático

# Eje Y con los valores AOQL incluidos
eje_y <- sort(unique(c(seq(0, max(aoql_vals)*1.2, length.out=5), aoql_vals)))
axis(2, at=round(eje_y, 4), labels=round(eje_y, 4), las=2, cex.axis=0.6)

# Normal
lines(pd, Pa_NII_Nor*pd,  lwd=2, lty=2, col=azules[2])
lines(pd, Pa_NIII_Nor*pd, lwd=2, lty=3, col=azules[3])

# Estricto
lines(pd, Pa_NI_Est*pd,   lwd=2, lty=1, col=rojos[1])
lines(pd, Pa_NII_Est*pd,  lwd=2, lty=2, col=rojos[2])
lines(pd, Pa_NIII_Est*pd, lwd=2, lty=3, col=rojos[3])

# Reducido
lines(pd, Pa_NI_Red*pd,   lwd=2, lty=1, col=verdes[1])
lines(pd, Pa_NII_Red*pd,  lwd=2, lty=2, col=verdes[2])
lines(pd, Pa_NIII_Red*pd, lwd=2, lty=3, col=verdes[3])
```

```r
# AOQL
abline(h=AOQL_NI_Nor,   lty=1, col=azules[1])
abline(h=AOQL_NII_Nor,  lty=2, col=azules[2])
abline(h=AOQL_NIII_Nor, lty=3, col=azules[3])
abline(h=AOQL_NI_Est,   lty=1, col=rojos[1])
abline(h=AOQL_NII_Est,  lty=2, col=rojos[2])
abline(h=AOQL_NIII_Est, lty=3, col=rojos[3])
abline(h=AOQL_NI_Red,   lty=1, col=verdes[1])
abline(h=AOQL_NII_Red,  lty=2, col=verdes[2])
abline(h=AOQL_NIII_Red, lty=3, col=verdes[3])

legend("topright",
       legend=c("Normal I","Normal II","Normal III",
                "Estricto I","Estricto II","Estricto III",
                "Reducido I","Reducido II","Reducido III"),
       col=c(azules, rojos, verdes),
       lwd=2, lty=c(1,2,3, 1,2,3, 1,2,3), cex=0.75)
grid()
```